Vyra Foundation — Requirements for a True Agentic GRC Platform

Vyra's capability specification — the model, the value, and the guarantees. This is the primary document: what Vyra is, whose job each layer serves, what compounds commercially, and *what must be technically true for a GRC system to earn the word "agentic."* Its technical references sit alongside it — `vyra-graph-spine.md` (the graph schema) and `vyra-architecture.md` (software layers and access rules). Build status is deliberately elsewhere: `vyra-tracker.md` for what's live, `vyra-implementation-plan.md` for what's next. Full doc map: `.design/README.md`.

> **Audience: founders, architects, and technical stakeholders.** The right-hand column of every table below is deliberately concrete — a claim about compliance that can't be stated as a structure is not a claim this document makes. Readers who want the pitch without the mechanism should be shown the two diagrams, not this file.

What Vyra Is

Vyra is an Agentic Risk & Compliance Infrastructure platform. AI agents continuously transform regulations into operational assurance by reasoning over a shared enterprise graph — the **Compliance Digital Twin**.

The operating loop, end to end:

```
Regulations → Requirements → Execution → Operations → Signals
```

That loop runs across five graph domains (full detail in `vyra-graph-spine.md`):

Graph	Question
Knowledge	What must be done?
Execution	What are we doing?
Operational	What is happening?
Intelligence	What do we understand?
Assurance	What can we prove?

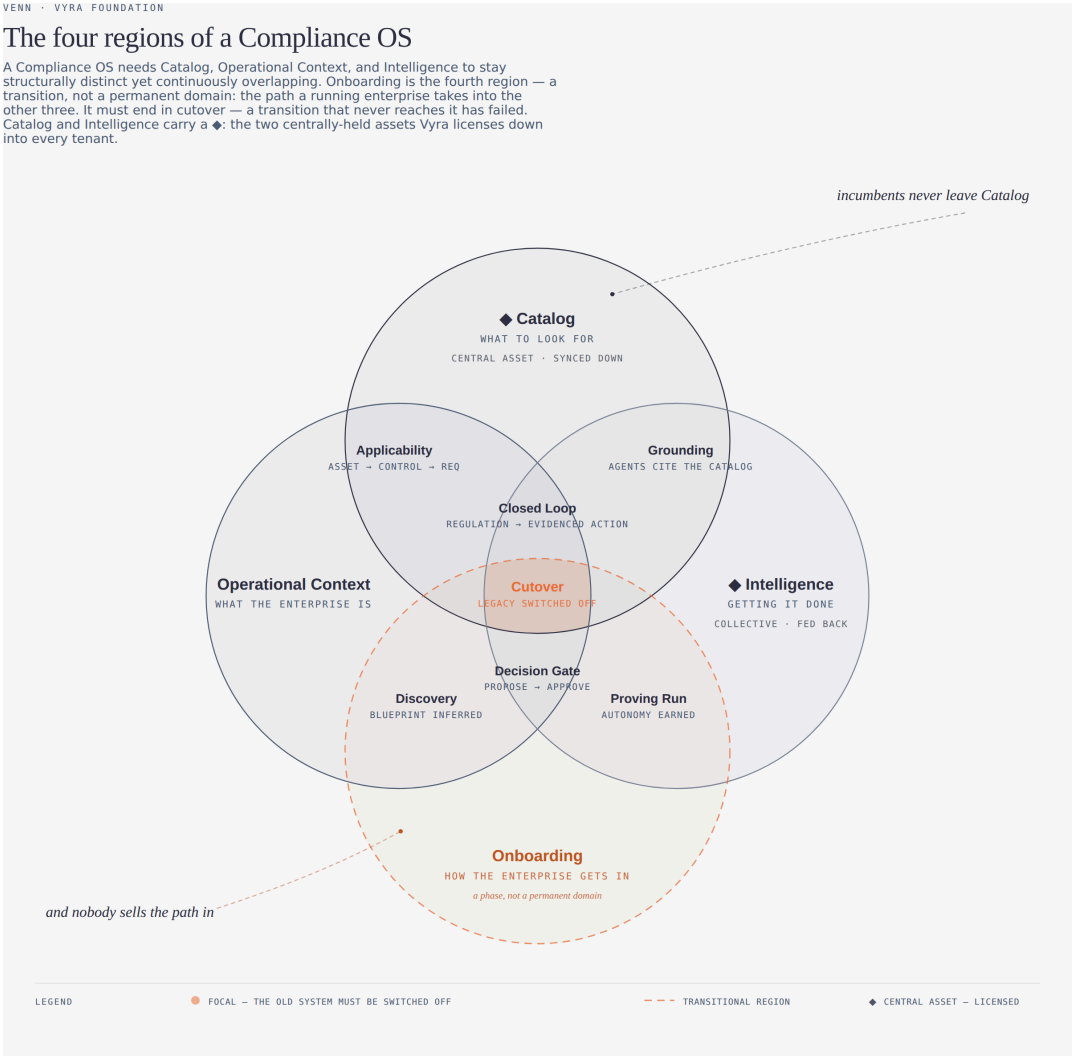
Vyra — Compliance. Handled.

The throughline

Every GRC product on the market today (ServiceNow GRC, OneTrust, LogicGate, Vanta, Drata, Archer) is a **document-and-workflow system of record**: policies live as text/PDF, obligation-to-control mapping is a human-maintained spreadsheet or form, "automation" means scheduled reminders and integration webhooks, and evidence is whatever a human uploads before an audit. None of them reason. None of them hold a live, queryable model of *what the enterprise actually is* connected to *what the law actually requires*, and none close the loop from a floor-level signal to a risk-scored, evidenced, human-approved action without a person manually walking every step.

What Vyra is building instead is a **Compliance OS for the business** — not a system that *records* the compliance function, but the system that function *runs on*. One substrate holds the obligations, the enterprise, and the reasoning; one system fills every role in the 7-layer operating model rather than handing each persona a better form to fill in. That is the ambition. Everything below is the set of guarantees that make it safe to hold.

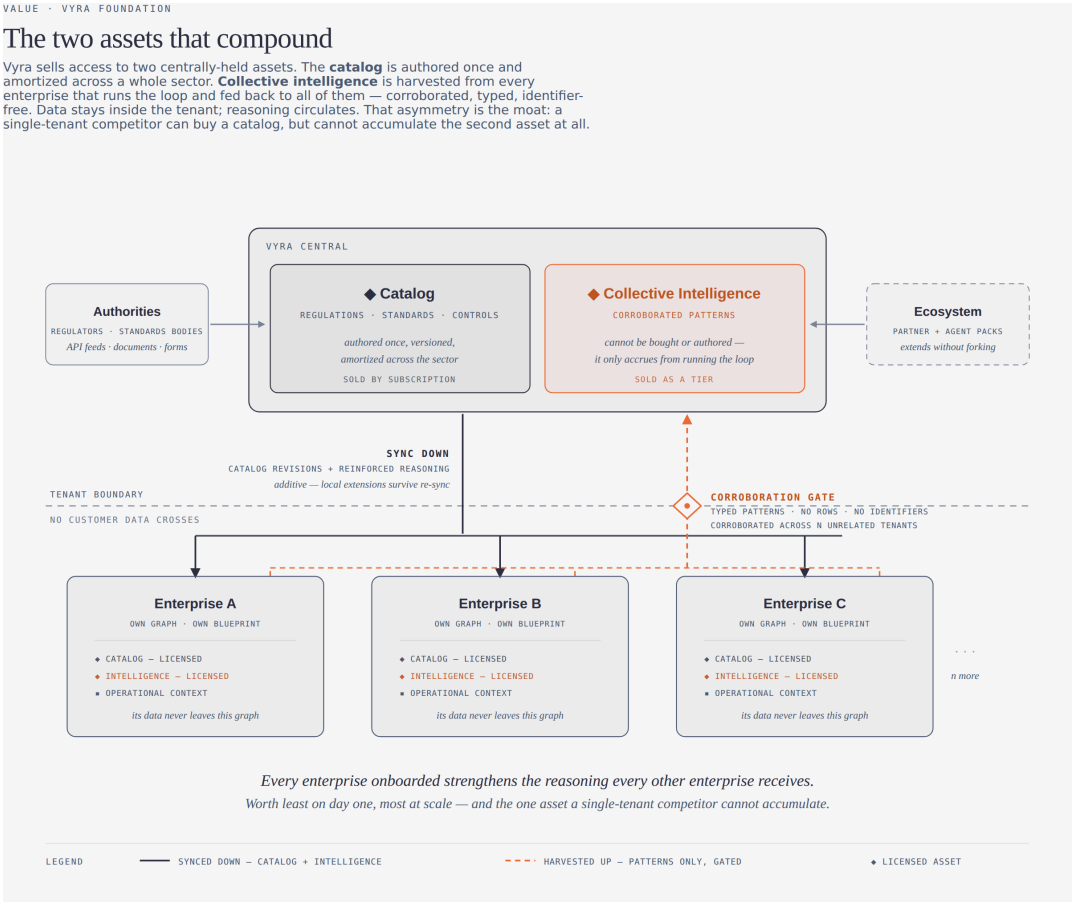
Agentic GRC means the system itself — not a human copying data between tools — performs the Observe → Interpret → Reason → Act → Verify → Learn cycle over a live graph, with autonomy that is explicit, leveled, and auditable rather than all-or-nothing. That requires three structurally distinct concerns to coexist without collapsing into one undifferentiated database — **and a fourth guarantee, orthogonal to the other three: that a running enterprise can *get into* that state without pausing its business.** A design that satisfies the three but only for a greenfield tenant hasn't solved GRC; it has solved a demo. Below is what each concern must guarantee, technically, to make that true — refined from the original Catalog / Operational Context / Intelligence framing and reconciled against Vyra's five-graph-domain design.



*Four regions. **Catalog**, **Operational Context** and **Intelligence** are the three permanent concerns, overlapping at Applicability, Grounding, the Decision Gate and — at their triple overlap — the **Closed Loop** that incumbent GRC tools never reach. **Onboarding** is the fourth, drawn dashed because it is a phase rather than a permanent domain: it meets Operational Context at **Discovery** and Intelligence at the **Proving Run**, and where all four meet (coral) is **Cutover** — the point at which legacy can be switched off on evidence. **Catalog** and **Intelligence** additionally carry a ◆ mark: they are the two centrally-held assets Vyra licenses down into every tenant. Interactive version.*

The two central assets — where the ecosystem's economics live

Vyra is a platform, not a product. The per-enterprise runtime is what an enterprise *operates*, but it is not what *compounds*. Two assets are held centrally, improve with every enterprise onboarded, and flow down into every tenant graph — and they are the two things Vyra actually sells. Both are marked on the diagram above; the mechanism between them and every tenant is below.



*The catalog is authored once and amortized across a sector. Collective intelligence is harvested from every enterprise that runs the loop and fed back to all of them — through a **corroboration gate** that admits only typed, identifier-free patterns seen across N unrelated tenants. Data stays in the tenant; reasoning circulates. Interactive version.*

Central asset	What it is	Why it compounds	What a tenant licenses
Catalog — regulatory knowledge	Versioned regulations, standards, clauses, requirements and reusable controls, maintained centrally as authorities publish revisions	Every jurisdiction, framework and revision added serves every enterprise in that domain — the maintenance cost is paid once and amortized across the sector	The base library by subscription, extensible locally without forking it (§1)
Collective Intelligence — reasoning	Corroborated patterns learned across enterprises in the same domain:	Every enterprise onboarded strengthens the reasoning every	A continuously reinforced Intelligence layer, without

	which mappings hold, which controls actually catch which signals, which risk patterns precede which findings	other enterprise receives — the one asset a single-tenant competitor structurally cannot accumulate	any tenant's data ever leaving its own graph (§3)

The catalog democratizes *what must be done*. Collective intelligence democratizes *how to do it well* — and it is by far the harder asset to copy, because it cannot be bought, licensed or authored; it only accrues from having run the loop across many enterprises, which means it is worth least on day one and most at scale. An enterprise buying Vyra is buying access to a sector's accumulated compliance reasoning, not a better place to keep its policies. A third surface — the ecosystem itself, third-party catalog packs, agent families and integrations — monetizes through the same extension points. **§4 sets out what must be technically true for any of the three to be sold, metered, entitled and defended.**

The Operating Model — seven layers, seven jobs

Sourced from the GRC Operating Model bow-tie. Each layer names a **persona** — who does the job today, and which agent family eventually absorbs it — and its **capabilities**, the jobs-to-be-done Vyra exists to serve.

> This table is the operating model, not a status report. For where each capability stands today (live / partial / gap) and what backs or blocks it, see `vyra-tracker.md`.

L	Layer	Persona → Agent Family	Capabilities
L1	**Knowledge**	Catalog Admin → Regulatory Intelligence Agents	Regulations · Standards · Contracts · SOPs
L2	**Interpret**	Ops Admin → Applicability Intelligence Agents	Applicability Scoping · Obligation Linkage
L3	**Planning**	Planner → Control Intelligence Agents	52-Week Calendar · Location + Role Assignment

L4	**CTN Knowledge Graph Spine** — Capture • Review	*(the graph itself — every agent's shared memory)*	The shared substrate all other layers read and write; "Review" = the Autonomy Level 1 human-approval gate
L5	**Oversight**	Ops Supervisor → Signal Intelligence Agents	Deviation Alerts · Escalation Paths
L6	**Assurance**	Compliance Mgmt → Assurance Agents	Coverage Scoring · Audit-Ready Export
L7	**Risk**	Risk Manager → Risk Intelligence Agents	Residual Risk Score · Scenario Simulation

Operating principles — the subset that governs day-to-day design decisions, not the full pitch:

- Agents collaborate **through the graph**, not through messaging — the graph is the operating system, not the product.
- Every compliance outcome must be traceable, **forward and reverse** (mechanics: `vyra-graph-spine.md`'s Graph Traversal Patterns).
- Every agent follows the same lifecycle: **Observe → Interpret → Reason → Act → Verify → Learn.**
- Not every action gets full autonomy. **Autonomy Levels 0–4** — Human Driven → Agent Recommends → Agent Assisted → Agent Executed → Fully Autonomous. Default is **Level 1 (Agent Recommends, Human Approves)** unless a specific workflow is explicitly elevated.

The JTBD Principle — what autonomy is actually optimizing for

The 7-layer operating model above names a persona and a Job To Be Done at every layer — Catalog Admin, Ops Admin, Planner, Ops Supervisor, Compliance Mgmt, Risk Manager. That JTBD framing isn't background context for a vision doc; it's the platform's actual optimization target. **The Intelligence layer exists to shrink each persona's JTBD, not to give them a better dashboard for doing it by hand indefinitely.**

A human belongs in the loop exactly where cognitive reasoning is required — a judgment call, a values tradeoff, an accountable sign-off that no amount of graph traversal resolves on its own. Everywhere else — scanning a regulation, mapping an obligation, scoring a routine risk, assembling evidence, drafting a recommendation — human involvement is a transitional cost the system is

designed to retire, not a permanent feature of the workflow. As an agent family's reasoning and provenance mature for a given class of decision, that class moves off the human's desk and onto commodity, agent-executed rails.

This is what the Autonomy Levels (0–4, above) actually measure: not "how much AI touches this workflow" but *how much of the persona's JTBD has already moved to commodity execution, and how little of what remains still needs a human's cognition.* **Level 1 (Agent Recommends, Human Approves) is the default because it's where an unproven workflow starts, not because it's where a mature one is meant to stay** — the target state for any given workflow is the least human cognition its risk profile actually requires, not universal Level 1 forever.

This is the same mechanism as onboarding. Migrating an enterprise off its legacy operating model (§0) and shrinking a persona's JTBD are not two capabilities — both are *reassigning a named job from a human actor to an agent actor at a chosen autonomy level* (§2, unified actor model). Onboarding is that operation performed the first time, workflow by workflow.

0. Onboarding — "how the enterprise gets in" (the transition guarantee)

> *Strategy is cheap. A friction-free transition into an agentic operating model, without business disruption, is what actually measures Vyra's success.*

Enterprises in the same line of business run the same regulations through wildly different plumbing — different site counts, role vocabularies, ownership splits, delegated processes, and a layer of undocumented human convention that is the real operating model. That convention is where friction lives, and it is invisible to any onboarding approach that starts from a questionnaire. Onboarding therefore has to produce two proofs, not one deliverable: that **the legacy system can be decommissioned** — every obligation it carried is carried here, its evidence chain reconstructable, nothing still depending on it — and that **what replaces it is a trusted partner, not a black box** — every action it takes attributable, reviewable, and reversible. Both are queries, not slides. That the business never stops is the constraint under which both are earned, not a third proof.

Cutover is not an aspiration of the engagement — it is the definition of its success. A transition that never reaches it has failed, however much value the platform delivered along the way, and that judgement belongs to Vyra rather than to the customer's comfort. The platform's obligation is to feed enough assurance to make the decommissioning decision provable; the enterprise's obligation is then to make it. The requirements below exist to make that outcome structurally hard to avoid.

Requirement	Technical shape

Parallel running is a proving phase that ends in a decommissioning decision	Legacy and Vyra run in parallel *deliberately and visibly*: Vyra observes real operations and produces its own tasks, mappings and decisions in **shadow mode** without being the system of record, while every proposal is scored against what the business actually did. The phase ends on a **queryable exit criterion** per workflow (agreement rate, obligation coverage, unresolved-gap count) — not on a date. Permanent coexistence is one failure mode; unmeasured cutover is the other
Cutover is committed at day zero, and lateness is an alarm state	Every workflow entering a proving run carries a target criterion *and* a window, agreed at onboarding and held as graph state. Past that window it becomes a first-class, queryable `cutover-overdue` state that names the specific blocking gaps, surfaced to the enterprise and to Vyra alike. A transition with no committed end is not a transition, and silence is precisely how parallel running becomes permanent
The proving run drives to the criterion — it doesn't merely observe	Onboarding agents are not passive scorers: their objective is to *close* the gaps blocking each workflow's cutover — assemble the missing evidence, resolve the unmapped obligations, raise the agreement rate on the classes where it lags. Assurance is manufactured deliberately until the decision becomes obvious
There is no exemption flag	A workflow that cannot cut over is a defect in Vyra's model, coverage or reasoning, routed back as product work — never a per-customer exemption setting. The moment an exemption exists as configuration, every hard workflow becomes one, and the platform has quietly re-accepted permanent coexistence
Decommission the operating model; an archive may remain, read-only	What must be switched off is the place where obligations are tracked, tasks assigned, and evidence assembled. A statutory archive may legitimately persist — but *read-only* is the test. If anyone still **works** in the old system, cutover has not happened, and "we have to keep it for retention" is the most common disguise for exactly that

A continuity baseline is captured before cutover, as graph data	"The business never needed to go back" is provable only against a recorded pre-Vyra baseline — the legacy obligation set, task cadence and completion rate, incident and escalation volumes, audit outcomes — persisted as nodes at day zero. Without a baseline, non-regression is an opinion, and the honest answer to "prove we don't need the old system" is that you can't
Decommissioning is an evidenced event, not an announcement	Switching the legacy system off is itself a `Decision` node with its evidence attached: a query showing every obligation it carried is now carried here, its historical evidence chain reconstructable end to end, and the continuity baseline met or beaten. The enterprise can then show a regulator *why* it was safe to switch off — the only version of this claim that survives contact with an auditor
Every enterprise onboards as a variant, not an exception	Shape differences (sites, role naming, hybrid ownership, outsourced steps, regional deviations) are **data on the blueprint** — properties and edges — never per-tenant code branches or bespoke schema. If accommodating an enterprise requires a code change, the model is wrong and the next enterprise will require another one
Onboarding is agent-run, not a services engagement	Discovery, mapping and blueprint assembly run the same Observe → Interpret → Reason → Act → Verify → Learn loop as every other agent family, under the same Decision gate. Onboarding cost per customer must therefore *fall* as those agents learn (§3) — a platform whose onboarding effort is flat is a consultancy with a graph attached
Cutover is per-workflow and reversible until its criterion is met	No big-bang date. Each workflow crosses independently, at its own autonomy level, when its own exit criterion fires; until then, returning that single workflow to human/legacy execution is a **state change on one node**, not a rollback project. Risk is bounded to one workflow at a time — but a reversal carries its own re-attempt criterion, so reverting is a loop back into the proving run, never an exit from it
Uninterrupted business is a	During transition, continuity is a live query with an alarm: no obligation left unassigned, no scheduled

monitored invariant, not a retrospective claim	control silently dropped, no task orphaned by a reassignment. A gap surfaced two weeks later has already disrupted the business

What this demands of the other three concerns: the catalog must accept the enterprise's regulatory material in whatever shape it already exists (§1); mapping must *infer* the enterprise rather than demand it be declared (§2); and autonomy must start low and be earned per workflow against measured evidence (§3).

1. Catalog — "what to look for" (Knowledge domain)

Requirement	Technical shape
Regulations, standards, obligations, controls are **first-class graph nodes**, not documents	`Regulation → Clause → Requirement`, polymorphic `Clause` parent (`Regulation` | `Standard`), reusable `Control` nodes independent of any one enterprise
Every input channel resolves to the same target shape	Authority API feeds, bulk document uploads (PDF/DOCX/HTML circulars), and hand-keyed form input all land as the same `Regulation → Clause → Requirement` nodes. The channel is recorded as provenance — it never becomes a parallel schema or a second-class "manual" partition
Unstructured text is *interpreted*, not merely parsed	Extracting an obligation from prose is a reasoning task with a confidence, not a regex pass: ingestion runs the full agent lifecycle, and a low-confidence extraction lands in an explicit `pending-interpretation` state for a human, rather than silently becoming a `Requirement` that the rest of the system then trusts
Source text is retained and addressable	Every `Clause`/`Requirement` keeps a pointer to its exact source span (document, version, anchor/offset). An auditor asking "where does this obligation come from" gets the clause text itself, not a citation string — and every past extraction can be re-litigated when the interpreting model improves
Catalog content is attributed interpretation, not axiom	Treating the catalog as "facts" is precisely the incumbents' brittleness: which regulation binds which entity type in which jurisdiction is a judgment. Store each such assertion with its author (human or agent),

	confidence and version, so it can be revised without rewriting history
Global and enterprise-specific catalogs coexist without merging identity	Structural dual-label (`:Catalog` vs `:Enterprise`) on the *same* node types — not a separate schema, not a property flag (a flag is invisible to pattern-matching and gets forgotten in queries/sync jobs)
One master, many synced copies — never a live cross-tenant query	One Neo4j graph per enterprise (no shared multi-tenant graph); a central versioned catalog store propagates down via a **periodic sync service**, chosen over live federation for query performance and blast-radius isolation
Every catalog fact is versioned and non-destructively superseded	`catalogVersion`, `effectiveFrom`, `supersededBy` per node — a repealed regulation is superseded, never deleted, so historical compliance state remains reconstructable
Queryable on any axis	Jurisdiction, authority, framework/standard, obligation type, control mechanism (preventive/detective/corrective), compliance area/domain taxonomy — each an independent traversal, not a hardcoded report
Enterprise customization survives re-sync (monetization surface)	Sync writes must be `MERGE ... ON MATCH SET n += row` (additive), so an enterprise can extend or annotate a catalog node with its own properties/links and not have them clobbered on the next central update — this is what makes the catalog tenant-monetizable (subscribe to the base library, extend it, don't fork it)
Freshness is a measurable SLA, not a promise	Every synced node/edge should be attributable to a sync run (timestamp, source revision) so "how current is our regulatory posture" is a query, not an assumption

2. Operational Context — "what the enterprise is, and what's happening" (Operational + Execution domains)

This is the hardest of the three. The catalog is the same for every enterprise in a sector; this is where they differ, and where onboarding either succeeds or turns into a consulting project.

Requirement	Technical shape
Enterprise structure is **graph, not free text**	`Organization → Role/Person → Facility → Asset`, `Vendor/Contract` — every entity a node with real relationships, so a compliance question can traverse from a regulation to the specific asset/role/site it binds to
The enterprise overlays the regulatory frame — never the reverse	The catalog stays canonical and structurally untouched; enterprise reality attaches to it through its own `:Enterprise` nodes and overlay edges. One enterprise's mapping choices can never mutate the shared frame, and a re-sync can never erase them (§1, additive `MERGE`). Direction of mapping is a load-bearing decision, not a stylistic one
Applicability is an explicit, inspectable edge, not implicit	`Asset -[:COVERED_BY]-> Control -[:IMPLEMENTS]-> Requirement` — a gap (an asset with no covering control) is a graph pattern, not a spreadsheet VLOOKUP
Operational workflow is captured as data, not described in prose	How a shift handover actually happens, who signs what, which check gates which — captured through structured capture (forms, imports, event streams, observation of live signal sequences) into first-class `Process → Step → Handoff` nodes. A workflow that exists only in a Word document cannot be orchestrated, monitored, or migrated, and is exactly where the undocumented human convention hides
The blueprint is inferred and proposed, never self-asserted	The system self-*builds* a candidate enterprise blueprint — org, roles, facilities, assets, processes, actors, and their mappings to obligations — as **agent proposals carrying confidence and rationale, ratified through the same Decision gate as any other agent action** (§3). "Self-building" that writes unratified structure straight into the system of record contradicts the gate and is the fastest way to lose an audit
The ratified blueprint is executable state, not a document	Once ratified, the blueprint *is* what the runtime orchestrates from: schedules, controls, tasks and assignments derive from it by query. If the blueprint has to be transcribed into a separate configuration before anything runs, it is a diagram, not an operating system
Humans and agents are the	`Actor` with `:Human` / `:Agent` labels (the same polymorphic idiom as `Clause`'s parent and the `:Catalog`/`:Enterprise` dual-label); `Task -`

same kind of assignable actor	`[:ASSIGNED_TO]-> Actor` resolves either way, and **the autonomy level is a property of the assignment, not of the platform.** This is what makes cutover a data change: moving a workflow off legacy is reassigning its tasks from human actors to agent actors, one workflow at a time, at a chosen level
Live operational signal, not a one-time snapshot	Floor/system events write directly into the graph as they happen (`Signal` nodes, direct API writes) — not a periodic CSV re-import — so "current posture" reflects today, not the last ingestion
The system can state its own posture at any instant	Coverage scoring, uncovered-requirement counts, and risk rollups must be computable as live queries over current graph state, not precomputed reports that go stale
Every mapping carries provenance	Was this Asset↔Control link asserted by ingestion, declared by a human, inferred by an agent, or observed from live behaviour? Confidence, origin, and timestamp must be attributes of the edge or an adjacent node — never lost by treating the mapping as "just true"
Documented absence is a first-class state	An asset category with no matching control ("unmapped," e.g. Security) must be queryable as a known gap, distinct from "not yet checked" — silent nulls are not acceptable in a system used for audit

3. Intelligence — "what needs to be done, and getting it done" (Intelligence + Assurance domains)

This is where the JTBD Principle above becomes mechanism — autonomy levels and the Decision Gate are how "shrink the JTBD, keep humans only for judgment" actually gets built, not just stated.

Requirement	Technical shape
A uniform agent lifecycle, not point automations	Every agent — regulatory, applicability, control, signal, risk, assurance, onboarding — runs the same loop: **Observe → Interpret → Reason → Act → Verify → Learn**. Different agent *families* own different sub-graphs; none call each other directly, none call the API — they coordinate only by reading/writing the shared graph
Autonomy is leveled and	Levels 0–4 (Human Driven → Agent Recommends → Agent Assisted → Agent Executed → Fully Autonomous),

explicit per workflow, never binary	default **Level 1** (agent proposes, human approves) unless a workflow is deliberately elevated. A "Decision" node with an approve/reject resolution path, attributable to a real person, is the structural gate — not a UI convention that can be bypassed
Autonomy is earned against measured evidence, and can be revoked	Elevating a workflow's level must be justified by its own record in the graph — agreement rate between agent proposals and human decisions for that class, over a real sample — computed as a query, not asserted in a config file. The same measure is what fires an onboarding exit criterion (§0), and a degrading rate must be able to demote a workflow as easily as it promoted it
Reasoning is tool-using and multi-step, not one prompt-in/JSON-out call	An agent that assembles one fixed prompt and parses the reply once is not reasoning — it's templated. A true agent must decide *what to look at next* in the graph across multiple turns before acting (this is the highest-risk, least commoditized capability — most "AI GRC" claims stop short of it)
Every autonomous action is traceable forward and reverse	From a regulation to the tasks/controls it generated, and from any finding/decision back to the specific clause, signal, and reasoning step that produced it — traceability is a graph-traversal guarantee, not a report generated after the fact
Evidence is generated, not collected	`EvidencePackage / Attestation / AssuranceStatement / Audit` as graph nodes derived from the actual chain of Requirement→Control→Signal→Decision — audit-ready export is a query over real provenance, not a document someone assembled the week before the audit
Risk and scenario reasoning require multiple agent families over shared state	A residual risk score or "what if this control fails" simulation isn't one agent's job — it needs risk-intelligence and control-intelligence (and potentially signal-intelligence) reasoning over the *same* live graph simultaneously. This is the capability that most separates "workflow AI" from genuinely agentic GRC, and the one competitors don't attempt
Learning closes the loop	Human overrides of agent proposals (rejections, corrections) must feed back into future reasoning for that agent family — otherwise "Learn" in the lifecycle is aspirational, not real, and onboarding cost never falls

Intelligence is democratized, not just the catalog	The second central asset. Reasoning learned across enterprises in the same domain — which mappings hold, which controls actually catch which signals, which risk patterns precede which findings — is harvested centrally and fed back down to strengthen every tenant's Intelligence layer. A single enterprise can only learn from its own history; the platform's advantage is that it learns from the sector's
What crosses the tenant boundary is defined by schema, not by policy	Only typed pattern objects — no free text, no identifiers, no rows — may leave an enterprise graph. The abstraction contract is enforced structurally, because "we promise not to send customer data" is not a posture that survives a security review, and this is the one place where the isolation guarantee could be quietly broken
Contribution is opt-in; feedback is not conditional on it	Whether an enterprise contributes to collective intelligence is an explicit, revocable per-tenant setting held as graph state. An enterprise that declines still receives the catalog and the platform — otherwise the isolation guarantee has a price tag attached, and it stops being a guarantee
A pattern must be corroborated before it feeds back	A behaviour observed in one tenant is that tenant's convention, not sector knowledge. Feedback requires corroboration across N unrelated enterprises with a recorded confidence — otherwise collective intelligence propagates one enterprise's bad habit to everyone, which is the same failure as agreement-rate conformity, scaled to the network

4. Value — "why anyone pays, and why anyone buys" (both sides of the ledger)

Not a business plan. The requirement is narrower and harder: ***both halves of the value exchange must be computable from the graph, not asserted in a deck.*** *If Vyra's revenue surfaces aren't structurally separable, and the enterprise's return isn't a live query, then this is a platform that only works when everyone is feeling generous.*

Requirement	Technical shape
Every monetizable asset is	Central catalog, central collective intelligence, the per-enterprise runtime, and third-party extensions must each be independently versioned and independently entitled —

structurally separable	so any one can be sold, priced, or withheld without forking the platform. An asset that can only be delivered as part of the whole cannot be monetized as a tier
Entitlement is data, never a build flag	What a tenant is licensed to receive — catalog scope, jurisdictions, agent families, collective-intelligence feedback — is graph state on that tenant, evaluated at sync and at runtime. A per-customer build is the point at which platform economics revert to services economics
Usage and value delivered are the same query	Metering (catalog nodes synced, patterns fed back, agent decisions executed) must derive from the same provenance the audit trail already requires. If billing needs its own instrumentation, the provenance model was incomplete
The ecosystem extends without forking	Third parties must be able to contribute catalog packs, agent families, and integrations through declared extension points that survive re-sync (§1's additive `MERGE`). A platform whose extensions require core changes is a product with a partner program bolted on
The continuity baseline is also the ROI denominator	The pre-cutover baseline captured in §0 — obligation coverage, task cadence and completion, incident and escalation volumes, audit outcomes — is not only the decommissioning proof. It is the *only* honest starting point against which return is measured. Both proofs run off the same nodes, which is why capturing it at day zero is non-negotiable twice over
Return is a live query, not a claimed number	Human decisions retired per workflow, obligations covered against baseline, signal-to-evidenced-action cycle time, audit-preparation effort, retired legacy licence and support cost — each computable over current graph state at any instant, and reconstructable at any past date (append-and-supersede)
One measure governs autonomy, cutover, and ROI	The JTBD Principle's measure — how much of a persona's job has moved to commodity execution — *is* the autonomy-elevation evidence (§3), *is* the cutover exit criterion (§0), and *is* the enterprise's return. Three claims off one instrument, or three instruments that will eventually disagree with each other in front of a customer
Value is attributable to a cause	Every retired job traces to the workflow and the autonomy elevation that retired it; every avoided finding traces to the control and signal that caught it. Aggregate ROI with no attribution chain is indistinguishable from a

coincidence, and will be treated as one by the person approving renewal

Cross-cutting non-negotiables

- **The graph is the only integration bus.** UI → API only; API/agents/ingestion → one shared data-access module → graph. No component reaches another directly. This is what lets Catalog, Operational Context, and Intelligence stay separately owned but always consistent.
- **Isolation of data, circulation of intelligence.** One graph per enterprise, not a shared multi-tenant store: no customer data traverses a boundary. What does circulate is *learned pattern* — corroborated, typed, identifier-free — flowing up into central collective intelligence and back down to every tenant. The distinction is the whole design: an enterprise gets the sector's accumulated reasoning without any enterprise's data ever leaving its own graph. Isolation is what makes the circulation sellable, not what limits it.
- **Exactly one system of record per workflow at any instant.** Vyra may run alongside a legacy system during transition, but which of the two is authoritative for a given workflow is an explicit, queryable property of that workflow. Two systems that both believe they are the source of truth is how compliance data quietly diverges — and it is the specific failure that makes a "temporary" parallel run permanent.
- **Cutover is mandatory.** Every workflow that enters a proving run is expected to leave it. Permanent parallel running is a failed engagement — not a customer preference to be accommodated, and not a status a workflow can rest in indefinitely. The platform is accountable for producing the assurance that makes the decommissioning decision safe and provable; an enterprise still working in its legacy system is the single clearest signal that Vyra has not done its job.
- **Nothing is deleted, only superseded.** Regulations, decisions, blueprints and evidence are append-and-supersede, never overwritten — a compliance system that can't reconstruct its own history at any past date has not solved compliance.
- **Every gap is a documented graph state, not a silent omission.** "Unmapped," "unscoped," "pending sync," "pending interpretation," "unratified" must be first-class, queryable states — an agentic system that can't say what it doesn't know isn't trustworthy enough to act autonomously.

Why this doesn't exist today

Incumbent GRC platforms automate *workflow* (tickets, reminders, approvals) around a document store. None of them run a shared, live, multi-agent reasoning

substrate over a structurally-versioned graph that spans regulation → enterprise structure → real-time signal → evidenced decision, with autonomy that is leveled rather than binary.

And none of them sell the transition. Incumbent onboarding is a services engagement: a partner interviews the business, hand-builds a control library and a spreadsheet of mappings, and the enterprise then runs both systems indefinitely — because nothing in the product ever *proves* the old one can be switched off. Vyra's claim is not only a better steady state; it's that the path into that state is agent-run, measured per workflow, and one-way by evidence rather than by decree — and that what the enterprise ends up with is not a GRC tool it operates alongside everything else, but the **Compliance OS its business runs on**.

That combination — not any single piece of it — is the product.